ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERIA DE SISTEMAS

Desarrollo de un aplicativo web y móvil de apoyo a la administración del Club de Bienestar Animal de la EPN

APLICACIÓN WEB DE CONTROL Y SEGUIMIENTO DE ADOPCIONES (BACKEND)

TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO COMO REQUISITO PARA LA OBTENCIÓN DEL TÍTULO DE INGENIERO EN SOFTWARE

Daniel Alejandro Vargas León

daniel.vargas02@epn.edu.ec

DIRECTOR: PABLO FERNANDO DEL HIERRO CADENA, MSc

pablo.delhierro@epn.edu.ec

DMQ, enero 2025

CERTIFICACIONES

Yo, DANIEL ALEJANDRO VARGAS LEON declaro que el trabajo de integración curricular aquí descrito es de mi autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que he consultado las referencias bibliográficas que se incluyen en este documento.

DANIEL ALEJANDRO VARGAS LEON

Certifico que el presente trabajo de integración curricular fue desarrollado por DANIEL ALEJANDRO VARGAS LEON, bajo mi supervisión.

PABLO FERNANDO DEL HIERRO CADENA
DIRECTOR

DECLARACIÓN DE AUTORÍA

A través de la presente declaración, afirmamos que el trabajo de integración curricular aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

AUTOR: DANIEL ALEJANDRO VARGAS LEON

DIRECTOR: PABLO FERNANDO DEL HIERRO CADENA

COLABORADORES:

BRAVO SANDOVAL DIANA CAROLINA

GONZALEZ GUANO RONNIE REYNEL

VENEGAS SANDOVAL JAIME GUSTAVO

VILAÑA TIPÁN MELANI SAMANTHA

DEDICATORIA

Con satisfacción dedico este trabajo de integración curricular a todas aquellas personas que forman una parte importante de mi vida por las cuales he podido cumplir con mis objetivos.

A mi madre, Elizabeth, por su cariño y su infaltable apoyo día con día. Por ser una de las personas las cuales día con día se preocupaban por mí y mi bienestar, procurando brindarme todo lo necesario para cumplir mis objetivos. Gracias a su esfuerzo constante y guía es quien soy en la actualidad. Le agradezco a la vida por ponerme a una madre como ella en mi vida.

A mi padre, Julio, por ser mi ejemplo de perseverancia, honestidad y rectitud. El cual, gracias a su apoyo constante, en cada una de las cosas que decidí, además de su apoyo en todo lo que me propongo, brindándome las herramientas o el apoyo necesario para lograr cumplir muchos de mis objetivos.

A mi abuelo, Jorge, el cual siempre me brindó una mano ton todo lo que necesitaba, buscando los modos. Igualmente, por su cariño y comprensión. Por ser un ejemplo de superación. Además de su forma de ser y sus enseñanzas, eh aprendido a ser una mejor persona. Gracias a él, a su curiosidad en temas técnicos y tecnológicos, por plantar las bases del camino que seguí.

A mi hermana, Adali, la cual con afecto y comprensión en todo lo que realizo fue un apoyo moral y emocional en mi vida.

A mis abuelos de parte Paterno, los cuales me demostraron un gran apoyo en mi día a día, Por el cariño que recibí de ellos día con día y la fe que me tuvieron, rogando por mi bienestar diariamente.

Daniel Vargas

AGRADECIMIENTO

Expreso mi agradecimiento a el magister Pablo de Hierro por brindarnos su apoyo desde el comienzo, guiándonos a lo largo de este proceso y por su paciencia. Gracias a su orientación y asesoría es que logramos que este trabajo se complete con éxito.

A todos mis familiares que me brindaros su apoyo de diversas maneras, para aquellos que estuvieron pendientes y buscaron ayudarme para lograr culminar, por cada una de sus palabras de aliento, su orgullo y confianza que me tuvieron es que logré culminar con este proyecto.

A mis amigos más cercanos, aquellos que la vida nos unió de forma inesperada y se quedaron conmigo. Para aquellos en los que deposité mi confianza. Agradezco su ayuda constante y apoyo moral en cada una de las situaciones en las que me eh encontrado a lo largo de este camino. Por brindarme Tardes y noches de alegría en las cuales eh podido desconectarme y despejar la mente.

A mis compañeros amigos y compañeros que conocí en la Universidad, aquellos compañeros y amigos de TIC, les agradezco por su apoyo y compañía en cada uno de los retos que se presentaron. Agradezco los momentos amenos que pasamos juntos y que hicieron que esta etapa universitaria sea inolvidable.

ÍNDICE DE CONTENIDO

CERTIFICACIONES I
DECLARACIÓN DE AUTORÍA II
DEDICATORIA III
AGRADECIMIENTO IV
ÍNDICE DE CONTENIDO V
RESUMEN VII
ABSTRACT VIII
1 DESCRIPCIÓN DEL COMPONENTE DESARROLLADO 1
1.1 Objetivo general 1
1.2 Objetivos específicos 1
1.3 Alcance 2
1.4 Marco teórico 2
2 METODOLOGÍA 8
2.1 Sprint 1 12
2.2 Sprint 2 18
2.3 Sprint 3 21
2.4 Sprint 4 24
2.5 Sprint 5 27
3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES 29
3.1 Resultados 29
3.2 Conclusiones 36
3.3 Recomendaciones 37
4 REFERENCIAS BIBLIOGRÁFICAS 38
5 ANEXOS 40
ANEXO I 41
ANEXO II 51

ÍNDICE DE FIGURAS

Figura 1. Primera versión del diagrama de clases. 13
Figura 2. Interfaz de Swagger. 16
Figura 3. Diagrama de Clases Actualizado con Visitas. 25
Figura 4. Diagrama de Clases actualizado. 28
Figura 5. Resultado del test de adopción de perro. 32
Figura 6. Resultado del test de adopción de perro con dueño existente. 32
Figura 7. Resultado de los test unitarios para el módulo de gestión de perros. 36

ÍNDICE DE TABLAS

Tabla 1. Herramientas utilizadas para el desarrollo backend 5
Tabla 2. Roles Scrum. 9
Tabla 3. Historias planteadas. 9
Tabla 4. Backlog de proyecto. 9
Tabla 5. Product Backlog estimado. 11
Tabla 6. HU “Eliminar Usuario”. 12
Tabla 7. Sprint Backlog 1. 12
Tabla 8. Sprint Backlog 2. 18
Tabla 9. Sprint Backlog 3. 21
Tabla 10. Sprint Backlog 4. 24
Tabla 11. Sprint Backlog 5. 27
Tabla 12. Story Points por Sprint. 29
Tabla 13. Historia de Usuario “Registrar perro adoptado”. 30

RESUMEN

El desarrollo de presente trabajo presenta el desarrollo de varios módulos, entro los cuales se encuentran la gestión de perros, gestión de usuarios, inicio de sesión para control de acceso, seguimiento de perros adoptados mediante visitas y la gestión de cursos e inscripciones. Este componente de la aplicación web promueve el bienestar animal mediante la tenencia responsable de animales de compañía, específicamente canes. Permitiendo y ayudando con una gestión eficiente de los perros a cargo del Club de Bienestar Animal, como lo son perros adoptados, permanentes y en espera de adopción.

Para el desarrollo de este componente se sigue la guía definida por el Marco de trabajo SCRUM, en forma de sprints y entregas funcionales que aporten valor. Ayudando a los interesados a tener ideas del avance y a los encargados del proyecto de posibles cambios y mejoras.

PALABRAS CLAVE: aplicación web, Bienestar Animal, club, Scrum.

ABSTRACT

The development of this work presents the development of several modules, among which are the management of dogs, user management, login for access control, monitoring of adopted dogs through visits and the management of courses and registrations. This component of the web application promotes animal welfare through responsible pet ownership, specifically dogs. Allowing and helping with an efficient management of the dogs in charge of the Animal Welfare Club, such as adopted dogs, permanent dogs and dogs awaiting adoption.

The development of this component follows the guidelines defined by the SCRUM Framework, in the form of sprints and functional deliverables that provide value. Helping stakeholders to have ideas of progress and project managers of possible changes and improvements.

KEYWORDS: web application, animal welfare, club, SCRUM.

1 DESCRIPCIÓN DEL COMPONENTE DESARROLLADO

El presente proyecto consiste en el desarrollo del backend de la aplicación web para el Club PoliPerros, cuyo objetivo es ofrecer una plataforma informativa y funcional que facilite la interacción de los usuarios con el club. Esta aplicación busca centralizar toda la información del club y mejorar la gestión de sus procesos internos.

Este componente ofrecerá una API para realizar el seguimiento de adopciones, se implementará una funcionalidad orientada a usuarios administradores. Este componente gestionará el almacenamiento de información sobre tutores responsables de animales adoptados, visitas de seguimiento de adopciones, estados e información de animales adoptados, nuevos ingresos y animales permanentes. Como también controlar la información referente a la promoción de cursos de adiestramiento que se oferten por parte del club. La gestión de datos aplicará técnicas de integridad y confidencialidad para garantizar la seguridad y privacidad de la información.

Mediante esta, se facilitará la comunicación fluida entre el cliente y el servidor, permitiendo el intercambio seguro y eficiente de dato. Esta también permitirá la integración con otros servicios y componentes externos, ampliando la funcionalidad y operatividad del sistema. Asegurando una escalabilidad y facilidad de mantenimiento del sistema, garantizando su estabilidad y rendimiento a largo plazo.

1.1 Objetivo general

Crear un sistema BackEnd que facilite la administración efectiva de la información relacionada con tutores, visitas de seguimiento, y estados de animales adoptados, permanentes y disponibles para adopción, garantizando la integridad y confidencialidad de los datos, y facilitando la comunicación fluida entre cliente y servidor mediante una API, con el fin de optimizar los procesos internos del Club de Bienestar Animal.

1.2 Objetivos específicos

1. Desarrollar un sistema de gestión que permita el control y seguimiento de animales ya adoptados.
2. Permitir el intercambio de datos entre sistemas para aumentar la interacción y escalabilidad.
3. Controlar y manejar la información relacionada al Club de Bienestar Animal.

4. Asegurar la confidencialidad e integridad de los datos mediante la implementación de medidas de seguridad.

1.3 Alcance

Este componente de la aplicación web realizará el seguimiento de adopciones para el Club de Bienestar Animal. Este incluye el desarrollo de un sistema Backend para gestionar la información de los tutores de los perros adoptados. Tener la información acerca de los perros que se encentren a cargo del Club de Bienestar Animal, como lo son los perros permanentes, aquellos que residen en la universidad, perros temporales, aquellos que están disponibles para adopción, y perros ya adoptados a los cuales se les podrá realizará el seguimiento mediante visitas.

También una Gestión de cursos, que incluye la creación, actualización, los cuales estarán disponibles para el público general. Además, con la capacidad de poder recibir inscripciones a estos mismos. Contar con módulos específicos para la gestión de cuentas de usuario, como la generación de nuevos usuarios (Administradora o auxiliar) y revocación de accesos de estos.

1.4 Marco teórico

A continuación, se exponen conceptos clave que ayudarán a entender la problemática. el backend del sistema, el marco de trabajo y las herramientas utilizadas para su desarrollo

Problemática de Fauna Urbana

En el distrito metropolitano de Quito el problema de fauna urbana es algo que se puede evidenciar en varios sectores de la capital. Donde los animales en situación de calle conllevan problemas de salud, como lo es la transmisión de plagas. En Quito existen alrededor de 1 perro en situación de calle por cada 19 habitantes, donde estos llegan a forma Jaurías que afectan a la seguridad. [1]

La forma de solucionar este problema de fauna urbana es promoviendo el bienestar animal y la tenencia responsable, con la cual se busca evitar el abandono y promover las adopciones. Así evitando problemas más comunes como ataques de canes, ataque a otros animales o desechos en las calles de la ciudad. [2]

Backend

Referencia programas y scripts que funcionan en un servidor, Es responsable de llevar a cabo las actividades vinculadas con el almacenamiento y recuperación de los datos,

conteniendo la lógica necesaria para el funcionamiento del de las aplicaciones web o un software en específico. Asegura que los datos solicitados por un Frontend u otro sistema se entreguen de la manera esperada.

Base de Datos (DB)

Es una recopilación de datos de forma sistemática, almacenados. Estos datos pueden ser de cualquier tipo, como números, cadenas de texto, binarios, etc. Las Bases de datos permiten recopilar una gran cantidad de información de forma estructurada que permiten un escalado eficiente de estos, manteniendo la integridad de los datos done estos no se van a deformar. En la actualidad estas se pueden almacenar en la nube. Además de existir diferentes tipos de bases de datos como las Relacionales y NoSQL, siendo su principal diferencia la de no usar relaciones tabulares, manteniendo un escalado horizontal. En cambio, una base de datos Relacional organiza sus datos en registros estructurados con tablas entrelazadas con formato de filas y columnas. [3]

Diagrama de Clases

Dentro del estándar de modela UML, el Diagrama de Clases permite la representación de objetos junto con sus atributos métodos y comportamiento. Estos permiten visualizar las clases dentro del sistema y las relaciones e interacciones con otras clases, estas usualmente usados en la programación Orientada a Objetos. Estos diagramas son importantes por su presentación gráfica y estructurada del sistema lo que ayuda a una mejor comprensión y guía en el desarrollo. [4]

Object Relational Mapping (ORM)

Las ORM permiten alinear el desarrollo del código con la estructura de las bases de datos, basado en metadata obtenida de los modelos creados. Esta herramienta esta altamente ligada a la programación orientada a objetos, buscando transformar esta a modelos relacionales de bases de datos. [5]

Las ORMs permiten la manipulación de objetos sin centrarnos completamente en diseños separados de POO y Modelos relacionales. Esta herramienta permite las operaciones CRUD (Create, Read, Update, Delete) sin necesidad de usar SQL, permitiendo ahorrar tiempo al evitar la escritura de consultas. [5]

Una de las ventajas de usar una ORM frente a código embebido con SQL es más eficiente de mantener la aplicación ya que permite actualizar toda la estructura de datos únicamente a partir del código y la ayuda de migraciones. Aunque una desventaja del uso de estas herramientas es su lentitud, debido al mapeo que tiene que realizar. También el diseño de

la base de datos puede llegar a ser pobre o ineficiente debido a una baja calidad de diseño en la programación orientada a objetos, por lo que, si el diseño previo de los modelos es ineficiente, las interacciones de las bases de datos igual.

Contenedores

Es un paquete de software ligero, los cuales se pueden ejecutar en cualquier entorno. Contienen todo lo necesario para la ejecución. Entre los cuales se puede obtener versiones concretas de entornos de ejecución. Los contenedores mantienen un aislamiento ya que se virtualizan los recursos, formando un entorno independiente del sistema host.

Una de sus principales ventajas en el desarrollo es la despreocupación por la compatibilidad entre sistemas, gracias a esta independencia y su facilidad de ejecución en casi cualquier entorno. Este empaquetamiento lógico permite un desarrollo ágil que optimiza la utilización de recursos. [6]

Pruebas unitarias

Las pruebas pueden mostrar la presencia de bugs dentro del programa, evaluar de manera individual el correcto funcionamiento de diversos componentes dentro del código. Esta es una técnica de Verificación y validación indispensable dentro de la ingeniería de software. Además, estos son usados para especificar los requerimientos, como el flujo de información que debería seguir. [7]

En frameworks de desarrollo existen diferentes tipos de librerías que facilitan la implementación y ejecución de pruebas unitarias

Scrum

Scrum es enfoque Ágil para desarrollar productos, se lo realiza con trabajo en equipo en pequeñas partes a la vez, con experimentación continua y ciclos de retroalimentación a lo largo del camino. Permitiendo aprender y mejorar a medida que avanza en el desarrollo del proyecto. Scrum ayuda a las personas y a los equipos a ofrecer valor de forma incremental y colaborativa. Como marco ágil, Scrum proporciona la estructura suficiente para que las personas y los equipos se integren en su forma de trabajar, al tiempo que agrega las prácticas adecuadas para optimizar sus necesidades específicas. [8]

El marco de Scrum es bastante simple y está formado por un equipo Scrum formado por un Product Owner, un Scrum Master y desarrolladores, cada uno de los cuales tiene responsabilidades específicas dentro del equipo. El Scrum Team participa en cinco eventos y produce tres artefactos. [8]

Scrum maneja valores, los cuales permiten que se lleve a cabo y con éxito. Los valores que Scrum busca son el Coraje, Enfoque, Compromiso, Respeto y Apertura, son fundamentales para los equipos Scrum al trabajar juntos. Son especialmente cruciales en entornos ya que experimentar, convivir y comunicarse es crucial para una mejora y aprendizaje continuo. Entre las principales ventajas que se tiene al optar por un Marco de trabajo como lo es Scrum es la Flexibilidad, la satisfacción del cliente ya que otorga valor continuamente a este y lo antes mencionado las entregas constantes que buscan las funcionalidades más importantes. Aunque optar por este marco de trabajo también tiene sus desventajas las cuales son su complejidad en un inicio para comprender su flujo de trabajo, además de perder el control en equipos de trabajo demasiado grandes ya que no se puede tener un éxito con el seguimiento y control del desarrollo.

Herramientas de desarrollo

Tabla 1. Herramientas utilizadas para el desarrollo backend

Nombre	Descripción	Versión	Aplicación
Lenguaje de programación			
Python	Python es un potente lenguaje de fácil entendimiento, con una gran comunidad y soporte. [9]	3.11	• Realizar la conexión con la base de datos y operaciones CRUD. • Proporcionar acceso a los datos a sistemas externos como el Frontend.
Herramientas de desarrollo			
Pycharm	Pycharm es un IDE enfocado en el desarrollo de Aplicaciones Web y Ciencia de datos, con compatibilidad para diferentes framework. [10]		• Crear el código fuente para el lado del servido, además del código para las pruebas unitarias.
XAMMP	Entorno usado para el desarrollo, el cual contiene una distribución gratuita de MySQL. [11]	8.2.4	• Levantar la Base de datos

GitHub	GitHub es una plataforma de gestión y organización en la nube basada en Git. Tienen la capacidad de seguir y administrar las modificaciones efectuadas en un código fuente a lo largo del tiempo. [12]	Online	• Gestionar las versiones del código. • Almacenar • Realizar la Integración y Despliegue continuo.

Fast Api

FastApi es un framework de desarrollo web enfocado en la construcción de APIs con Python. Este framework presenta una facilidad de uso, especialmente cundo se trata de APIs RESTful. Gracias a la integración de starletter y pydantic se tiene una validación para los tipos de datos. Además de ayudar a la reducción de Bugs. [13]

FastApi presenta herramientas que permite la generación de documentación automática mediante Swagger y ReDocs. Entre las principales características que presenta el framework es la validación de datos, haciendo que sea uno de los frameworks más rápidos en Python. Además de poder soportar programación asíncrona con las funciones async y await, mejorando el rendimiento. [13]

2 METODOLOGÍA

El Desarrollo del este componente backend fue llevado a cabo con el marco de trabajo Scrum. Se planificaron 5 Sprints con duración de 4 semanas y jornadas de 5 horas diarias. Se detallan las principales etapas de proceso:

1. Preparación Inicial

   En esta etapa se habló con el Product Owner para comprender las necesidades y problemas que busca resolver mediante la aplicación. Se partió de un prototipo para tener mejor idea de lo que se busca lograr. Se definieron los roles dentro del equipo de trabajo y se establecieron objetivos a alcanzar.

2. Planificación y Estimación

   En esta etapa se consolidaron las ideas obtenidas en la etapa inicial como historias de usuario. Se asigno a cada una de estas una estimación de esfuerzo y prioridad. Estas actividades de grooming permiten identificar aquellas actividades más complejas y urgentes de completar, permitiendo priorizar el desarrollo en aquellas con mayor valor e impacto.

3. Implementación

   En esta etapa de planifican y ejecutan los Sprint, buscando tener una entrega funcional y de alto valor para el cliente al final de cada uno de estos. Además de realizar modificaciones según las necesidades del usuario.

4. Revisión y retrospectiva

   Al final de cada sprint se realizó un análisis de trabajo realizado para evaluar que se hizo bien y que no. Esto permite mejorar continuamente, optimizando el trabajo a realizar o redefinirlo, de ser necesario, para el siguiente sprint.

5. Lanzamiento

   Se entrega el producto final al cliente. En esta se obtiene la aprobación final de su parte, validando los requisitos propuestos al inicio del proyecto.

Preparación Inicial

En esta fase se llevó a cabo una reunión con la administradora del Club de Bienestar Animal PoliPerros, la Ingeniera María Fernanda Orquera junto con el magíster Pablo del

Hierro y el equipo de trabajo. El propósito de dicha reunión es levantar requisitos y definir los objetivos y alcance de la aplicación.

Una vez entendidos los requerimientos y flujos de la aplicación se lleva a cabo el siguiente proceso definido por el marco de trabajo Scrum. Se llevaron a cabo las siguientes actividades.

Roles de Scrum

Se definieron los siguientes roles según el marco de trabajo:

Tabla 2. Roles Scrum.

Rol	Responsable
Product Owner	María Fernanda Orquera
Scrum Master	Pablo del Hierro MSc.
Desarrolladores	Daniel Alejandro Vargas León Melani Samantha Vilaña Tipán

Levantamiento de Requisitos

A partir del levantamiento de requisitos realizado junto al equipo se identificaron las siguientes historias épicas según las necesidades del proyecto.

Tabla 3. Historias planteadas.

Identificador	Título
GC	Módulo para la Gestión de Cursos
IS	Módulo de Sesión y Autenticación
GP	Módulo para la Gestión de Perros
VS	Módulo de Visitas y Seguimiento
CP	Módulo de Configuración Personal
GU	Componente para la Administración de Usuarios
GI	Componente para la Administración de Inscripciones

Product Backlog

Los requerimientos de la aplicación se formalizaron mediante historias de usuario que se encuentran en la Tabla 4.

Tabla 4. Backlog de proyecto**.**

Historia Épica	Identificador	HU
Módulo para la Gestión de Cursos (GC)	GC-1	Creación de Cursos
	GC-2	Obtener información de Cursos
	GC-3	Actualizar Cursos
	GC-4	Eliminar Cursos
Módulo de Sesión y Autenticación (IS)	IS-1	Generación de Token
	IS-2	Correo para la recuperación de la contraseña
Módulo de Gestión de Perros (GP)	GP-1	Registrar perro permanente
	GP-2	Registrar perro temporal
	GP-3	Registrar perro adoptado
	GP-4	Consultar lista de perros permanentes
	GP-5	Consultar lista de perros temporales
	GP-6	Consultar lista de perros adoptados
	GP-7	Actualizar el registro del perro permanente
	GP-8	Actualizar información de perro temporal
	GP-9	Actualizar información de perro adoptado
	GP-10	Eliminar registro de perro permanente
	GP-11	Eliminar registro de perro temporal
	GP-12	Eliminar registro de perro adoptado
	GP-13	Consultar perros en base a su id
Módulo de Visitas y Seguimiento (VS)	VS-1	Registrar visita
	VS-2	Obtener lista Visitas
	VS-3	Obtener visitas por Perro
	VS-4	Actualizar visita
	VS-5	Eliminar Visita
	VS-6	Actualizar información de dueño
Módulo de Configuración Personal (CP)	CP-1	Atualizar credenciales
Componente para la Administración de Usuarios (GU)	GU-1	Generar Usuario
	GU-2	Obtener Usuarios Creados
	GU-4	Eliminar Usuarios
Componente para la Administración de Inscripciones (GI)	GI-1	Inscripciones por curso
	GI-2	Eliminar inscipciones

Planificación y estimación

Se establecieron prioridades y se realizó la estimación de las historias de usuario considerando su prioridad y complejidad. Esto permite determinar la importancia de cada una de estas historias dentro del proyecto, Los resultados están disponibles en la Tabla 5.

Tabla 5. Product Backlog estimado.

Identificador	Historia de Usuario	Prioridad	Estimación
GC-1	Creación de Cursos	Media	5
GC-2	Obtener información de Cursos	Media	3
GC-3	Actualizar Cursos	Media	5
GC-4	Eliminar Cursos	Media	3
IS-1	Generación de Token	Alta	8
IS-2	Correo para la recuperación de la contraseña	Media	5
GP-1	Registrar perro permanente	Alta	5
GP-2	Registrar perro temporal	Alta	5
GP-3	Registrar perro adoptado	Alta	5
GP-4	Consultar lista de perros permanentes	Alta	3
GP-5	Consultar lista de perros temporales	Alta	3
GP-6	Consultar lista de perros adoptados	Alta	3
GP-7	Actualizar el registro del perro permanente	Alta	5
GP-8	Actualizar información de perro temporal	Alta	5
GP-9	Actualizar información de perro adoptado	Alta	8
GP-10	Eliminar registro de perro permanente	Media	3
GP-11	Eliminar registro de perro temporal	Media	3
GP-12	Eliminar registro de perro adoptado	Media	5
GP-13	Consultar perros en base a su id	Alta	5
VS-1	Registrar visita	Alta	8
VS-2	Obtener lista Visitas	Alta	3
VS-3	Obtener visitas por Perro	Alta	3
VS-4	Actualizar visita	Alta	5
VS-5	Eliminar Visita	Media	3
VS-6	Actualizar información de dueño	Baja	5
CP-1	Actualizar credenciales	Baja	3
GU-1	Generar Usuario	Media	5
GU-2	Obtener Usuarios Creados	Baja	3

GU-3	Eliminar Usuarios	Baja	5
GI-1	Inscripciones por curso	Media	8
GI-2	Eliminar inscripciones	Baja	5

Historias de Usuario

Se definieron historias de usuario junto con sus criterios de aceptación, lo que permite tener una visión más clara acerca de los objetivos a cumplir con cada una de estas. Los criterios funcionan como métricas las cuales permiten evaluar y garantizar el resultado de la historia de usuario. La Tabla 6 presenta un ejemplo ilustrativo de una historia de usuario, mientras que las demás se incluyen en el Anexo I.

Tabla 6. HU "Eliminar Usuario".

HU	
Identificador: GU-4	**Usuario**: Administrador
Nombre: Eliminar Usuario	
Prioridad: Baja	**Riesgo de desarrollo:** Bajo
Estimado: 5	**Iteración:**
Descripción: Como Administradora quiero eliminar usuarios creados o generados para que ya no tengan acceso a los módulos de gestión.	
Criterios de Aceptación: - No se debe eliminar el usuario actual de la sesión. - Se podrá eliminar un usuario. - El usuario eliminado no podrá volver a iniciar sesión.	

2.1 Sprint 1

El objetivo del primer Sprint es implementar la seguridad y el acceso a él back mediante un flujo de usuario, incluyendo la autenticación y generación de un token y recuperación de la contraseña. Además del registro y obtención de los perros permanentes y temporales, estableciendo la estructura de la información.

Para alcanzar el objetivo establecido, se llevó a cabo la planificación de sprint. La Tabla 7 proporciona un detalle de las HU empleadas para este propósito.

Tabla 7. Sprint Backlog 1.

Identificador	Historia de Usuario
IS-1	Generación de Token

IS-2	Correo para la recuperación de la contraseña
GP-1	Registrar perro permanente
GP-2	Registrar perro temporal
GP-4	Consultar lista de perros permanentes
GP-5	Consultar lista de perros temporales

Al concluir el primer Sprint se completaron todas las tareas relacionadas con estas historias de usuario. Como resultado se tiene una primera aproximación de la estructura de la aplicación mediante un diagrama de clases. Además de los endpoints para autenticación, y operaciones de Creación y Lectura de perros Temporales y Permanentes. La Figura 1 muestra el diagrama de clases resultante.

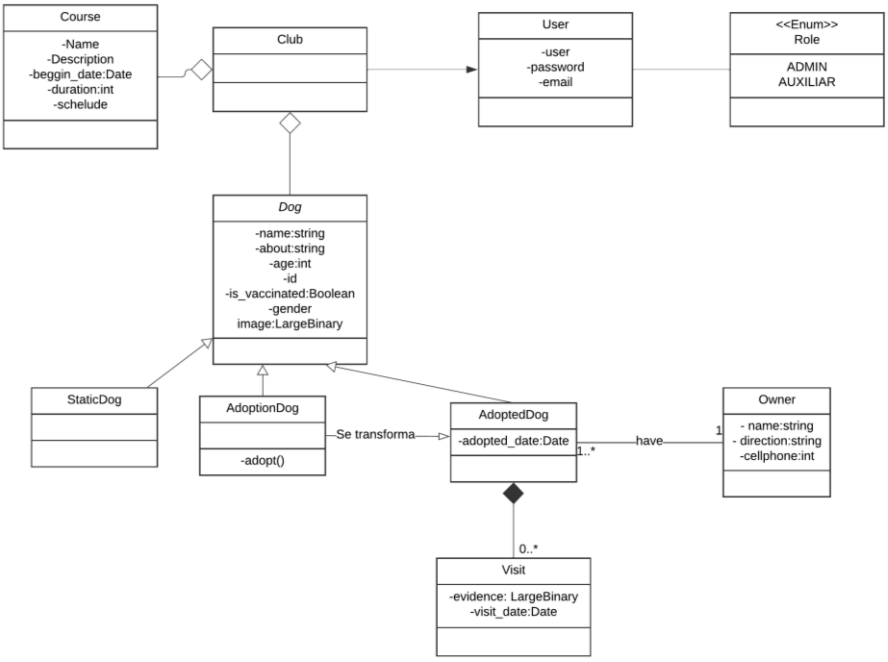

Figura 1. Primera versión del diagrama de clases.

Implementación

- **Historia de usuario GP-1**

A continuación, se presenta el código desarrollado para el endpoint enfocado en la creación de un perro permanente de la historia de usuario “GP-1: Registrar perro permanente”.

```
@router.post('/static_dog/create/', response_model=dict)
async def create_new_static_dog(dog: StaticDogCreate,
                                db: Session = Depends(get_db),
                                current_user: TokenData =
Depends(get_current_user)):
    """
    English:
    --------
    Create a static dog:

    - **id_chip** (optional): chip of the dog.
    - **name** (required): Name of the dog.
    - **about** (optional): Description of the dog.
    - **age** (required): Age of the dog.
    - **is_vaccinated** (required): Indicates if the dog is vaccinated.
Must be a boolean:
        - **true**: The dog is vaccinated.
        - **false**: The dog is not vaccinated.
    - **image** (optional): Base64 encrypted image of the dog.
    - **gender** (required): Gender of the dog. Must be one of:
        - **male**: Represents a male dog.
        - **female**: Represents a female dog.
    - **entry_date** (optional): Date of the entry in format YYYY-MM-DD.
    - **is_sterilized** (required): Indicates if the dog is sterilized.
Must be a boolean:
        - **true**: The dog is sterilized.
        - **false**: The dog is not sterilized
    - **is_dewormed** (required): Indicates if the dog is dewormed. Must
be a boolean:
        - **true**: The dog is dewormed.
        - **false**: The dog is not dewormed
    - **operation** (optional): Specify the operation of the dog.

    Español:
    --------
    Crear un perro estático:

    - **id_chip** (optional): Chip del perro.
    - **name** (required): Nombre del perro.
    - **about** (optional): Descripción del perro.
    - **age** (required): Edad del perro.
    - **is_vaccinated** (required): Indica si el perro esta vacunado.
Debe ser un boolean:
        - **true**: El perro esta vacunado.
        - **false**: El perro no esta vacunado.
    - **image** (optional): Imagen del perro encriptad en base64.
    - **gender** (required): Genero del perro. Debe ser uno de los
siguientes:
        - **male**: Representa un perro macho.
        - **female**: Representa un perro hembra.
    - **entry_date** (optional): Fecha de entrada en formato YYYY-MM-DD.
    - **is_sterilized** (required): Indica si el perro esta esterilizado.
Debe ser un boolean:
        - **true**: El perro esta esterilizado.
        - **false**: El perro no esta esterilizado.
    - **is_dewormed** (required): Indica si el perro esta desparasitado.
```

```
Debe ser un boolean:
        - **true**: El perro esta desparasitado.
        - **false**: El perro no esta desparasitado.
    - **operation** (optional): Especifica la/las operaciones del perro.
    """

    if current_user.role.value not in [Role.ADMIN, Role.AUXILIAR]:
        raise HTTPException(status_code=403, detail="Not enough
permissions")
    image_data = None
    if dog.image:
        try:
            image_data = base64.b64decode(dog.image)
        except (ValueError, TypeError):
            raise HTTPException(status_code=400, detail="Invalid image
encoding")
        # verificamos el tamaño de la imagen
    if image_data:
        try:
            verify_image_size(image_data)
        except ValueError:
            raise HTTPException(status_code=400, detail="Invalid image
size")

    # Crear el perro en la base de datos
    result = create_static_dog(db, dog, image_data)
    if result is None:
        raise HTTPException(status_code=404, detail="Ya existe")
    return result
```

El código correspondiente a las demás HU vinculadas a este sprint está disponible en el Anexo II.

Se generó la “documentación viviente” mediante el uso de Swagger, la cual permite interactuar con el funcionamiento de los endpoints. Mostrando una ventana como la presentada en la Figura 2, donde se pude observar la estructura de la información a ser enviada por el cuerpo de la solicitud, un ejemplo de este y el Método por el cual se debe enviar.

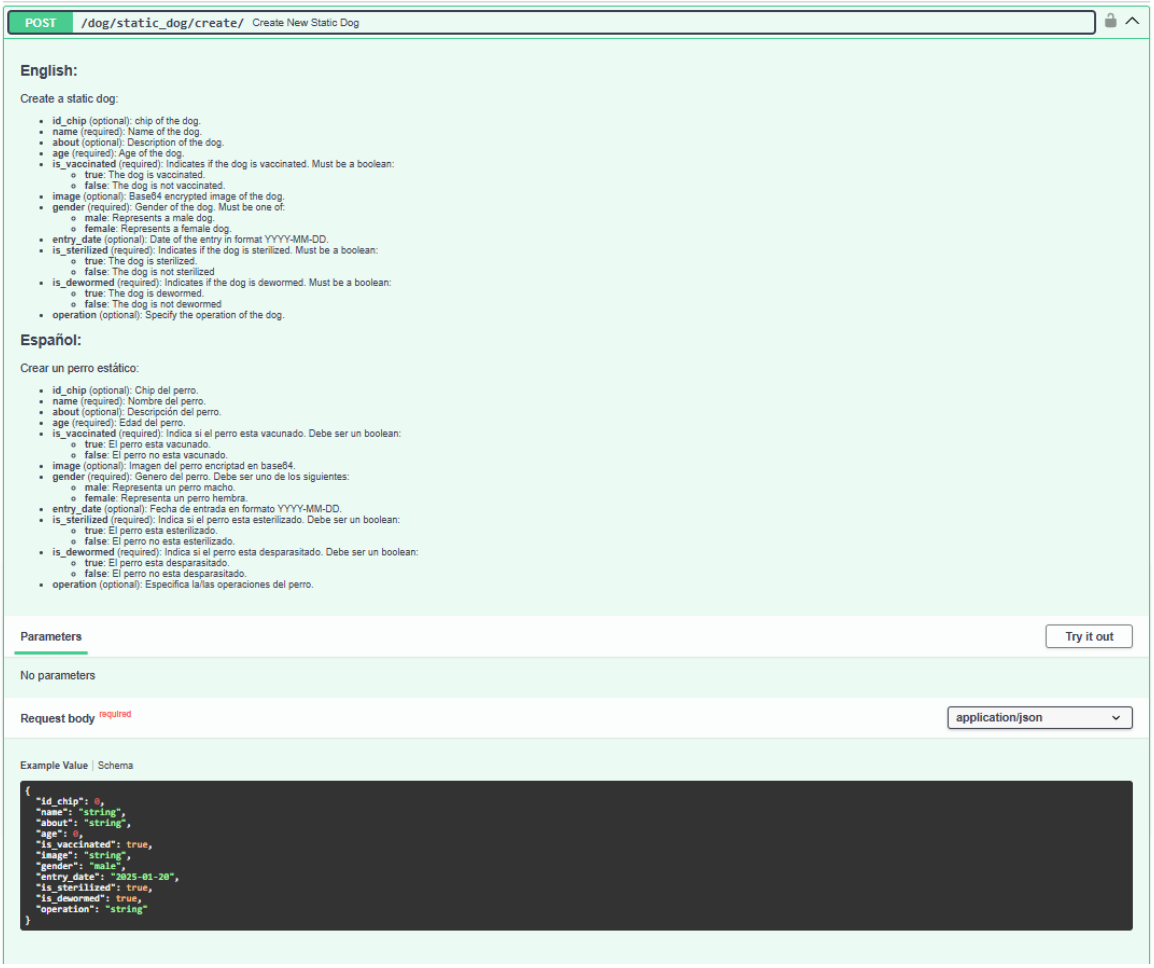

Figura 2. Interfaz de Swagger.

Es fundamental incluir un DocString en cada uno de los métodos los cuales proporcionan documentación extra en la interfaz de Swagger acerca de la información que este recibe, como el tipo de dato, o si se debe enviar un parámetro de una enumeración.

Para una Integración y Despliegue continuo se hizo uso de las Github Actions las cuales permiten la Dockerización para una integración continua y la actualización de un a aplicación contenedora de Azure para el despliegue continuo. A continuación, se presenta el código usado para la implementación de la herramienta de GitHub.

```
name: CI Pipeline

on:
  push:
    branches: ["master"]
  pull_request:
    branches: ["master"]
```

```yaml
jobs:
  build-and-push-docker:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v3
        with:
          python-version: '3.9'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Build Docker image
        run: |
          docker build -f docker/Dockerfile -t poliperritos .

      - name: Log in to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_HUB_USERNAME }}
          password: ${{ secrets.DOCKER_HUB_ACCESS_TOKEN }}

      - name: Push Docker image
        run: |
          docker tag poliperritos ${{ secrets.DOCKER_HUB_USERNAME
}}/poliperritos:${{ github.sha }}
          docker push ${{ secrets.DOCKER_HUB_USERNAME }}/poliperritos:${{
github.sha }}

  deploy-to-azure:
    needs: build-and-push-docker
    runs-on: ubuntu-latest
    steps:
      # Azure CLI Login
      - name: Log in to Azure CLI
        uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      # Update Container App with New Image
      - name: Update Azure Container App
        run: |
          az containerapp update \
            --name poliperritosback \
            --resource-group ${{ secrets.AZURE_RESOURCE_GROUP }} \
            --image ${{ secrets.DOCKER_HUB_USERNAME }}/poliperritos:${{
github.sha }}
```

Cabe aclarar que esta es una configuración única durante este sprint para el CI y CD, ya que se ejecutará estas acciones con cada actualización del código dentro del repositorio.

Sprint Review

En el sprint se completaron varias historias de usuario clave. Se implementó la seguridad de los endpoints y restricción basada en roles. Además del flujo para el inicio de sesión y Correo para la recuperación de la contraseña (IS-1, IS-2). Así como las funcionalidades para registrar y obtener los perros temporales y permanente (PG-1, PG-2, PG-4, PG-5). Estas funcionalidades fueron revisadas y aprobadas por el equipo con comentarios de mejora y ajustes para los próximos Sprints.

Retrospectiva del sprint

Al final del sprint se llevó a cabo una evaluación detallada del trabajo, analizando los siguientes puntos:

- **¿Qué se hizo bien?** Las historias de usuario se completaron enfocadas en el objetivo previsto para el sprint, cumpliendo con los requisitos esperados.
- **¿Qué se hizo mal?** Se presentaron retrasos en la integración debido a la falta de control de excepciones las cuales no fueron manejadas adecuadamente.
- **¿Qué se puede mejorar?** Se puede mejorar el manejo de excepciones ya que estos son comunes y repetitivos, además de mejorar la comunicación con el equipo acerca de los detalles de la funcionalidad de los diversos endpoints generados.

2.2 Sprint 2

El objetivo del segundo sprint está enfocado en las funcionalidades más importantes de los perros, que son la lectura actualización y eliminación de perros temporales y permanentes, además de la creación de un perro adoptado, la cual su lógica es pasar de un perro temporal a uno adoptado y la eliminación de este, la cual su lógica es pasar de perro adoptado a perro temporal.

Para alcanzar el objetivo establecido, se llevó a cabo la planificación de sprint. En la Tabla 8 se detallan las historias de usuario usadas para este.

Tabla 8. Sprint Backlog 2.

Identificador	Historia de Usuario
GP-3	Registrar perro adoptado
GP-6	Consultar lista de perros adoptados
GP-7	Actualizar el registro del perro permanente

GP-8	Actualizar información de perro temporal
GP-10	Eliminar registro de perro permanente
GP-11	Eliminar registro de perro temporal
GP-12	Eliminar registro de perro adoptado

Al concluir el sprint se concretó todas las tareas asociadas a las historias de usuario planteadas. Como resultado de las historias seleccionadas, tenemos una iteración la cual aporta gran valor al usuario, ya que las principales funciones de adopción ya se encuentran implementadas, además de la consulta de los perros adoptados deja listo el panorama para continuar con las siguientes historias, relacionadas con visitas, en el siguiente sprint.

- **Historia de usuario** GP-3

A continuación, se presenta el código de la implementación del endpoint enfocado en la adopción de un perro. Para el cual se busca que un usuario con permisos pueda realizar este proceso, se necesita registrar al dueño o responsable de perro y el perro en cuestión a ser adoptado. A continuación, se presenta el código desarrollado para la ejecución de la HU “GP-3 Registrar perro adoptado”:

```
@router.post('/adoption_dog/adopt/{dog_id}/{adoption_date}',
response_model=dict)
def adopt_dog_by_id(dog_id: int, adoption_date: date, owner: OwnerCreate,
db: Session = Depends(get_db),
                    current_user: TokenData = Depends(get_current_user)):
    """
    English:
    --------
    Adopt a dog that is available for adoption:

    - **dog_id** (required): id of the adoption dog to be adopted.
    - **adoption_date** (required): Date of the adoption in format YYYY-
MM-DD.

    - **name** (required): Name of the owner.
    - **direction** (required): Direction of the owner.
    - **cellphone** (required): Cellphone of the owner.

    Español:
    --------
    Adoptar a un perro que esté disponible para adopción:

    - **dog_id** (required): id del perro de adopción.
    - **adoption_date** (required): Fecha de la adopción en formatoYYYY-
MM-DD.

    - **name** (required): Nombre del dueño.
    - **direction** (required): Dirección del dueño.
```

```
    - **cellphone** (required): Teléfono del dueño.
    """
    if current_user.role.value not in [Role.ADMIN, Role.AUXILIAR]:
        raise HTTPException(status_code=403, detail="Not enough
permissions")
    adoption_dog = read_adoption_dog_by_id(db, dog_id)

    if adoption_dog is None:
        raise HTTPException(status_code=404, detail="No existe")
    adopted_dog = adoption_dog.adopt(adoption_date, owner)
    result = create_owner_and_adopted_dog(db, adopted_dog)
    return result


@router.post('/adoption_dog/adopt/{dog_id}/{owner_id}/{adoption_date}',
response_model=dict)
def adopt_dog_by_id_and_existing_owner(dog_id: int, adoption_date: date,
owner_id: int,
                                       db: Session = Depends(get_db),
                                       current_user: TokenData =
Depends(get_current_user)):
    """
    English:
    --------
    Adopt a dog that is available for adoption:

    - **dog_id** (required): id of the adoption dog to be adopted.
    - **adoption_date** (required): Date of the adoption in format YYYY-
MM-DD.
    - **owner_id** (required): Id of an existing owner.

    Español:
    --------
    Adoptar a un perro que esté disponible para adopción:

    - **dog_id** (required): id del perro de adopción.
    - **adoption_date** (required): Fecha de la adopción en formatoYYYY-
MM-DD.
    - **owner_id** (required): Id de un Dueño existente.
    """
    if current_user.role.value not in [Role.ADMIN, Role.AUXILIAR]:
        raise HTTPException(status_code=403, detail="Not enough
permissions")
    adoption_dog = read_adoption_dog_by_id(db, dog_id)
    if adoption_dog is None:
        raise HTTPException(status_code=404, detail="Perro de adopción no
existe")
    owner = read_owner_by_id(db, owner_id)
    if owner is None:
        raise HTTPException(status_code=404, detail="Dueño no existe")
    adopted_dog = adoption_dog.adopt_existing_owner(adoption_date, owner)
    result = adopt_dog(db, adopted_dog)
    return result
```

El código correspondiente a las demás HU vinculadas a este sprint está disponible en el Anexo II.

Así mismo la “Documentación viviente” generada por Swagger puede ser encontrada en la ruta /docs.

Sprint Review

En el sprint se completaron varias historias de usuario clave para una de las funcionalidades más importantes del sistema, que en este caso son las adopciones. Cada una de las historias de usuario completas cumplen con los criterios de aceptación previstos en las historias de usuario correspondientes (GP-3, GP-6, GP-7, GP-8, GP-10, GP-11, GP-12).

Retrospectiva del sprint

Al final del sprint se llevó a cabo una evaluación detallada del trabajo, analizando los siguientes puntos:

- **¿Qué se hizo bien?** Las historias de usuario se completaron enfocadas en el objetivo previsto para el sprint, cumpliendo con los requisitos esperados.
- **¿Qué se hizo mal?** Se presentaron problemas con la implementación de las imágenes las cuales en un principio se optó por almacenarlas en base 64 lo cual ralentizaba la obtención de las imágenes asociadas, de igual forma en base64.
- **¿Qué se puede mejorar?** Se puede mejorar el manejo de imágenes para devolver una url con el acceso de las imágenes para evitar el transporte de una gran cantidad de caracteres por la conversión a base 64. Evitando el aumento del 33% de las imágenes también en el retorno de estas.

2.3 Sprint 3

El objetivo del Tercer Sprint es la funcionalidad enfocada en la culminación del módulo de gestión de perros y la generación de usuarios y configuración de perfiles. Entre las cuales se encuentran funcionalidades como generar usuarios ya sean administradores o Auxiliares, editar las credenciales de acceso del perfil personar y revocación de acceso a personar ya no autorizado.

Para alcanzar el objetivo establecido, se llevó a cabo la planificación del sprint. En la tabla 8 se detallan las historias de usuario correspondientes a este sprint.

Tabla 9. Sprint Backlog 3.

Identificador	Historia de Usuario

GP-9	Actualizar información de perro adoptado
VS-6	Actualizar información de dueño
CP-1	Cambiar contraseña
GU-1	Generar Usuario
GU-2	Obtener Usuarios Creados
GU-3	Lista de usuarios
GU-4	Eliminar Usuarios

Al concluir el sprint se concretó todas las tareas asociadas a las historias de usuario planteadas. Como resultado de las historias seleccionadas, tenemos una iteración la cual permite la generación de usuarios, la actualización de credenciales de acceso personales y además de la edición de información del dueño.

Implementación

- **Historia de usuario G**U-1

A continuación, se presenta el código de la implementación del endpoint enfocado en la generación de un usuario (Administradora o auxiliar). Para el cual se busca que únicamente un usuario Administradora podrá realizar este proceso, se necesitará un correo al cual se enviarán las credenciales de acceso y se asociará dicho correo con la cuenta para procesos de Correo para la recuperación de la contraseña. A continuación, se presenta el código desarrollado para llevar a cabo la implementación de la HU“GU-1 Generar Usuario”:

```
@router.post("/generate_user", response_model=dict)
def generate_new_auth_user(background_tasks: BackgroundTasks,
                           email: str,
                           role: Role,
                           db: Session = Depends(get_db),
                           current_user: TokenData =
Depends(get_current_user)):
    """
       English:
       --------
       Generate a new User:

       - **email** (required): New email.
       - **role** (required): Role of the user. Must be one of:
            - **admin**: An admin user.
            - **auxiliar**: An auxiliar user.

       Español:
       --------
       Generar un nuevo usuario:

       - **email** (required): Correo electrónico.
       - **role** (required): Rol del usuario. Debe ser uno de los
```

```
siguientes:
            - **admin**: Usuario administrador.
            - **auxiliar**: Usuario auxiliar.
        """
    if current_user.role.value not in [Role.ADMIN]:
        raise HTTPException(status_code=403, detail="Not enough
permissions")
    if not verify_email(email):
        raise HTTPException(status_code=400, detail="Correo inválido")
    user, data_for_email = user_generator(email, role)
    auth_user = auto_create_auth_user(db, user)
    if not isinstance(auth_user, HTTPException):
        background_tasks.add_task(send_email, email, "Credenciales de
acceso", data_for_email, "user.html")
        return auth_user
    else:
        return auth_user
```

El Código relacionado con las otras HU consideradas para este sprint está disponible en el Anexo II.

Así mismo la “Documentación viviente” generada por Swagger puede ser encontrada en la ruta /docs.

Sprint Review

En el sprint se completaron varias historias de usuario clave la funcionalidad clave en la seguridad del sistema, que en este caso son la gestión de cuentas de acceso. Cada una de las historias de usuario completas cumplen con los criterios de aceptación previstos en las historias de usuario correspondientes (GP-9, VS-6, CP-1, GU-1, GU-2, GU-3, GU-4).

Retrospectiva del sprint

Al final del sprint se llevó a cabo una evaluación detallada del trabajo, analizando los siguientes puntos:

- **¿Qué se hizo bien?** Las historias de usuario se completaron enfocadas en el objetivo previsto para el sprint, cumpliendo con los requisitos esperados. Aumentando la seguridad del sistema y permitiendo la configuración de acceso de cada usuario, manteniendo confidencial los datos importantes como las credenciales de acceso.

- **¿Qué se hizo mal?** Se presentaron problemas con la implementación de la actualización de perros adoptados, esto debido al funcionamiento de el ORM SqlAlchemy, el cual actualiza el objeto de la sesión en la base de datos

automáticamente generando problemas, al no encriptar de datos, al momento de sincronizar los nuevos cambios

- **¿Qué se puede mejorar?** Se debe manejar correctamente la sesión, además de especificar y colocar correctamente en el flujo del programa la encriptación y desencriptación de datos para evitar una mala persistencia en la base de datos.

2.4 Sprint 4

El propósito del cuarto Sprint es implementar la funcionalidad de Visitas., la cual es una de las principales dentro del sistema. La cual se va a poder obtener ya sea todas, todas las visitas por perro además de actualizarlas y eliminarlas. También se enfoca en obtener la información concreta de un perro en específico.

Para alcanzar dichos objetivos, se llevó a cabo La planificación del sprint se detalla en la Tabla 9, donde se especifican las historias correspondientes a este sprint.

Tabla 10. Sprint Backlog 4.

Identificador	Historia de Usuario
GP-13	Consultar perros en base a su id
VS-1	Registrar visita
VS-2	Obtener lista Visitas
VS-3	Obtener visitas por Perro
VS-4	Actualizar visita
VS-5	Eliminar Visita

Al concluir el sprint, se completaron todas las tareas asociadas a las historias de usuario. Como resultado de esta iteración se tiene una aplicación más concreta que aporta valor al usuario. Gracias a la implementación, casi en su totalidad de los dos módulos principales, de Gestión de perros y el Módulo de Visitas y seguimiento. Además, se creó una nueva versión del diagrama de clases, con cambios en los modelos de Visitas.

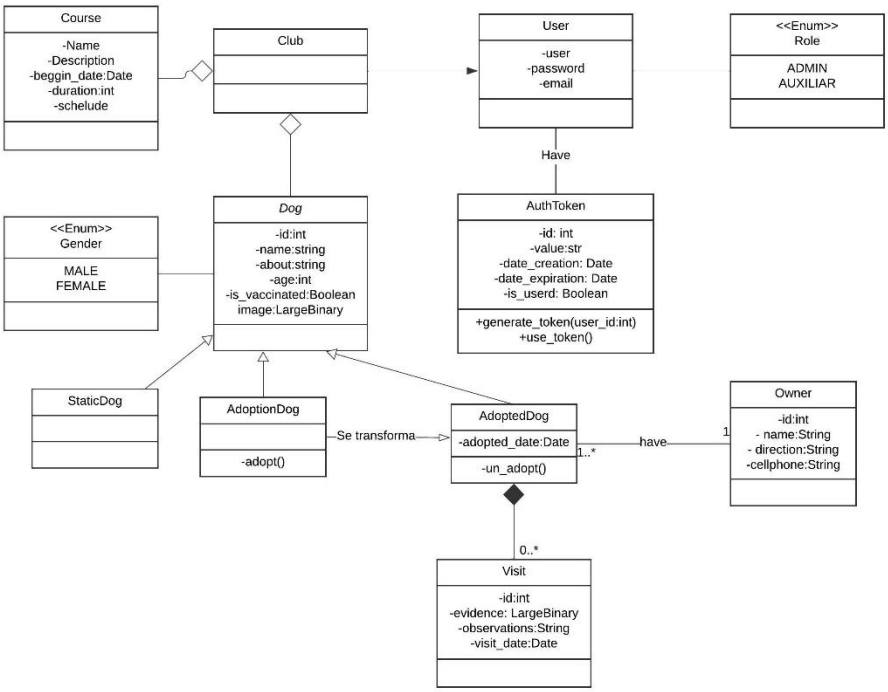

Figura 3. Diagrama de Clases Actualizado con Visitas.

Implementación

- **Historia de usuario** VS-1

A continuación, se presenta el código de la implementación del endpoint enfocado en el registro de una visita para un perro. Para lo cual se debe obtener la información relevante como evidencia, fecha de la visita y observaciones. A continuación, se presenta el código desarrollado para la HU "VS-1 Registrar visita":

```
@router.post('/create/', response_model=dict)
async def create_new_visit(visit: VisitCreate,
                           db: Session = Depends(get_db),
                           current_user: TokenData =
Depends(get_current_user)):
    """
    English:
    --------
    Create a visit for an adopted dog:

    - **visit_date** (required): date of the visit in format YYYY-MM-DD.
    - **evidence** (optional): base64 encrypted image of a evidence of
the visit.
    - **observations** (optional): any observation of the visit.
    - **adopted_dog_id** (required): id of the dog visited.

    Español:
    --------
    Crear una visita para un perro adoptado:
```

```
    - **visit_date** (required): fecha de la visita en formato YYYY-MM-
DD.
    - **evidence** (optional): imagen encriptada en base 64 de la visita.
    - **observations** (optional): observación de la visita.
    - **adopted_dog_id** (required): id del perro visitado.
    """

    if current_user.role.value not in [Role.ADMIN, Role.AUXILIAR]:
        raise HTTPException(status_code=403, detail="Not enough
permissions")
    image_data = None
    if visit.evidence:
        try:
            image_data = base64.b64decode(visit.evidence)
        except (ValueError, TypeError):
            raise HTTPException(status_code=400, detail="Invalid image
encoding")
    # verificamos el tamaño de la imagen
    try:
        verify_image_size(image_data)
    except ValueError:
        raise HTTPException(status_code=400, detail="Invalid image size")

    # Obtenemos el perro
    adopted_dog = read_adopted_dogs_by_id(db, visit.adopted_dog_id)
    if not adopted_dog:
        raise HTTPException(status_code=404, detail=f'No se encontró al
perro con id: {visit.adopted_dog_id}')
    adopted_dog.owner.crypt_owner_data()
    result = create_a_visit(db, visit, adopted_dog, image_data)
    if result is None:
        raise HTTPException(status_code=409, detail="La visita que desea
registrar ya existe")
    return result
```

El Código relacionado con las otras HU consideradas para el para este sprint está disponible en el Anexo II.

Así mismo la “Documentación viviente” generada por Swagger puede ser encontrada en la ruta /docs.

Sprint Review

En el sprint se completaron varias historias de usuario clave para el sistema, ya que es una de las funcionalidades centrales de este, como lo son el módulo de Visitas y seguimiento. Con esto se completaron las historias de mayor prioridad para el desarrollo y que más aportan valor al sistema. Cada una de las historias de usuario completas cumplen con los criterios de aceptación previstos en las historias de usuario correspondientes (GP-13, VS-1, VS-2, VS-3, VS-4, VS-5).

Retrospectiva del sprint

Al final del sprint se llevó a cabo una evaluación detallada del trabajo, analizando los siguientes puntos:

- **¿Qué se hizo bien?** Las historias de usuario se completaron enfocadas en el objetivo previsto para el sprint, logrando completar los módulos de mayor importancia.
- **¿Qué se hizo mal?** Se presentaron problemas en el control de errores y la carga de imágenes ya que estas no se presentaban como debían y al actualizar una visita se eliminaba la imagen.
- **¿Qué se puede mejorar?** Se puede optimizar la carga de imágenes al reducir el tamaño máximo que se puede aceptar ya que aún se las sigue recibiendo con base 64.

2.5 Sprint 5

El ultimo sprint está enfocado en la gestión de cursos y de las inscripciones de este. Permitiendo crear nuevos cursos, visualizarlos, editarlos y eliminarlos. Además de poder recibir inscripciones en estos y controlar las mismas.

Para alcanzar el objetivo establecido, se llevó a cabo la planificación de Sprint. En la tabla 10 se detallan las historias de usuario correspondientes a este Sprint.

Tabla 11. Sprint Backlog 5.

Identificador	Historia de Usuario
GC-1	Creación de Cursos
GC-2	Obtener información de Cursos
GC-3	Actualizar Cursos
GC-4	Eliminar Cursos
GI-1	Inscripciones por curso
GI-2	Eliminar inscripciones

Al concluir el sprint, se completaron todas las tareas asociadas a las historias de usuario definidas en la planificación del sprint. Como resultado se implementó el módulo de Gestión de Cursos y la Gestión de Inscritos. Con esto completando los últimos requerimientos propuestos. Permitiendo una gestión de los cursos en las cuales se tienen horarios personalizables y un control de alumnos que se pueden inscribir mediante la implementación de cupos por curso. Además, se actualizó el diagrama de clases enfocado en los modelos de Curso y Aplicante.

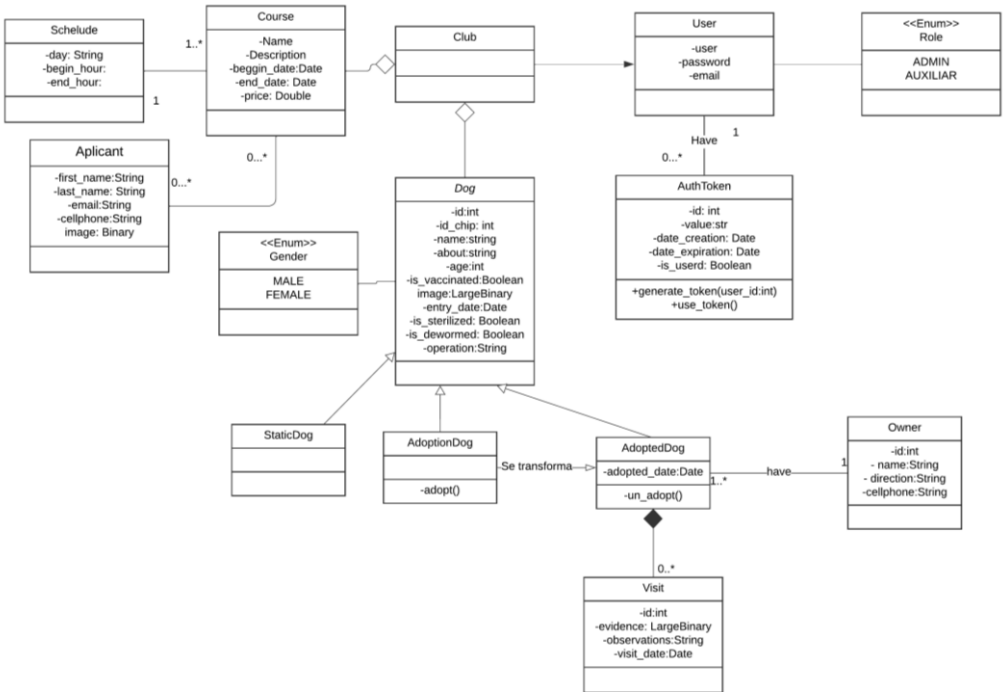

Figura 4. Diagrama de Clases actualizado.

Implementación

- **Historia de usuario** GC-2

A continuación, se presenta el código de la implementación del endpoint enfocado en la obtención de la información de los cursos, ya sea obteniendo todos los cursos o algún curso en específico. A continuación, se presenta el código desarrollado para llevar a cabo la implementación de la HU “GC-2 Obtener información de los cursos”:

```
@router.get('/', response_model=List[CourseResponse])
def get_all_courses(db: Session = Depends(get_db)):
    response = read_all_course(db)
    if not response:
        raise HTTPException(status_code=404, detail="No se encontraron
cursos")
    return response


@router.get('/{id_course}', response_model=CourseResponse)
def get_course_by_id(id_course: int,
                     db: Session = Depends(get_db)):
    response = read_course_by_id(db, id_course)
    if not response:
        raise HTTPException(status_code=404, detail="No se encontraron
cursos")
    return response
```

El Código relacionado con las otras HU consideradas para este sprint está disponible en el Anexo II.

Así mismo la “Documentación viviente” generada por Swagger puede ser encontrada en la ruta /docs.

Sprint Review

En el sprint se finalizaron las historias de usuario centradas en la creación., actualización y eliminación de cursos. Permitiendo controlar su capacidad y aspectos importantes como la descripción, su precio y horarios. Además de permitir la inscripción de personas mediante una solicitud que se almacenará en la base de datos. Cada una de las historias de usuario completas cumplen con los criterios de aceptación previstos en las historias de usuario correspondientes (GC-1, GC-2, GC-3, GC-4, GI-1, GI-1).

Retrospectiva del sprint

Al final del sprint se llevó a cabo una evaluación detallada del trabajo, analizando los siguientes puntos:

- **¿Qué se hizo bien?** Se completaron todas las HU planificada. Logrando implementar las diferentes funcionalidades esperadas de gestión de cursos e inscritos.
- **¿Qué se hizo mal?** Se presentaron problemas en la forma en que se almacenan las solicitudes de inscripción.
- **¿Qué se puede mejorar?** Se puede optimizar la forma en que se obtiene la información en concreto de los inscritos, organizada por curso o por todas las inscripciones.

3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES

3.1 Resultados

Para concluir el proyecto, se llevaron a cabo 5 Sprints con un total 143 story points, el detalle de cada uno se presenta en la Tabla 10.

Tabla 12. Story Points por Sprint.

Sprint	Story points
1	29

2	29
3	27
4	29
5	29

Para verificar y validar el desarrollo de los componentes, se hizo uso de test unitarios y test de integración. Mediante los cuales podremos validar los criterios de aceptación. Para el cual, con la historia de usuario GP-3 presente en la Tabla 11.

Tabla 13. Historia de Usuario "Registrar perro adoptado".

Historia de usuario	
Identificador: GP-3	**Usuario:** Administrador
Nombre: Registrar perro adoptado	
Prioridad: Alta	**Riesgo:** Alta
Estimado: 5	**Iteración:** 3
Descripción: Como administradora quiero registrar un perro adoptado para mantener un registro actualizado de los perros disponibles de adopción de la universidad.	
Criterios de aceptación: -se debe registrar la fecha de adopción, nombre del adoptante, dirección y teléfono y un id de perro estático. - Se puede registrar una adopción con un dueño existente. - Se debe eliminar el perro temporal. - Se debe persistir el perro adoptado junto a su dueño	

Donde se generó los siguientes test de integración relacionados a la historia de usuario.

```
def test_adopt_dog_by_id():
    setup_db()
    create_auth_user_for_test()
    adoption_dog = create_adoption_dog_for_tests()

    adoption_dog_db =
next(override_get_db()).query(AdoptionDog).filter(AdoptionDog.id ==
adoption_dog).first()
    assert adoption_dog_db is not None
    token_response = client.post(
        "/auth/token",
        data={"username": "admin", "password": "SecurePassword123"}
    )
    assert token_response.status_code == 200
    token = token_response.json()["access_token"]
    response = client.post(f"/dog/adoption_dog/adopt/{adoption_dog}/2025-
01-01",
                           headers={"Authorization": f"Bearer {token}"},
                           json={
                               "name": "Luis",
                               "direction": "Quitumbe",
                               "cellphone": "0979040404"
                           })
    assert response.status_code == 200
```

```
    assert "Perro Adoptado." == response.json().get("detail")
    adoption_dog_db =
next(override_get_db()).query(AdoptionDog).filter(AdoptionDog.id ==
adoption_dog).first()
    assert adoption_dog_db is None
    adopted_dog_db =
next(override_get_db()).query(AdoptedDog).filter(AdoptedDog.id ==
adoption_dog).first()
    assert adopted_dog_db is not None
    adopted_dog_db.owner.decrypt_owner_data()
    assert adopted_dog_db.owner.name == "Luis"
    teardown_db()


def test_adopt_dog_by_id_and_existing_owner():
    setup_db()
    create_auth_user_for_test()
    adoption_dog = create_adoption_dog_for_tests()
    db = next(override_get_db())
    existing_owner = Owner(id=1, name="Luis", direction="San Bartolo",
cellphone="0998899876")
    existing_owner.crypt_owner_data()
    db.add(existing_owner)
    db.commit()
    adoption_dog_db =
next(override_get_db()).query(AdoptionDog).filter(AdoptionDog.id ==
adoption_dog).first()
    assert adoption_dog_db is not None
    token_response = client.post(
        "/auth/token",
        data={"username": "admin", "password": "SecurePassword123"}
    )
    assert token_response.status_code == 200
    token = token_response.json()["access_token"]
    response =
client.post(f"/dog/adoption_dog/adopt/{adoption_dog}/{existing_owner.id}/
2025-01-01",
                            headers={"Authorization": f"Bearer {token}"})
    assert response.status_code == 200
    assert "Perro Adoptado creado" == response.json().get("detail")
    adoption_dog_db =
next(override_get_db()).query(AdoptionDog).filter(AdoptionDog.id ==
adoption_dog).first()
    assert adoption_dog_db is None
    adopted_dog_db =
next(override_get_db()).query(AdoptedDog).filter(AdoptedDog.id ==
adoption_dog).first()
    assert adopted_dog_db is not None
    assert adopted_dog_db.owner.id == existing_owner.id
    teardown_db()
```

De la historia de usuario se obtuvieron 2 test, los cuales se valida la adopción creando un dueño y la adopción cuando un dueño ya existe. Además, se colocó un assert par cada uno de los otros puntos a validar en los criterios de aceptación como registro de la información del usuario, la eliminación del perro temporal y la persistencia del dueño en la base de datos.

Los demás test de integración se encuentran en el Anexo II.

Al ejecutar los test de integración relacionados con la historia de usuario de la Tabla 13 se obtienen los resultados para el test de adopción y adopción con un dueño existente en la Figura 5 y la Figura 6 respectivamente. En estos resultados se como los criterios de aceptación para la historia de usuario propuesta se superaron satisfactoriamente sin errores. Lo que indica que la funcionalidad propuesta a sido implementada correctamente.

```
============================= test session starts ==============================
collecting ... collected 1 item

test_dog.py::test_adopt_dog_by_id PASSED                                 [100%]

======================= 1 passed, 14 warnings in 2.14s =======================

Process finished with exit code 0
```

Figura 5. Resultado del test de adopción de perro.

```
============================= test session starts ==============================
collecting ... collected 1 item

test_dog.py::test_adopt_dog_by_id_and_existing_owner PASSED              [100%]

======================= 1 passed, 14 warnings in 2.13s =======================

Process finished with exit code 0
```

Figura 6. Resultado del test de adopción de perro con dueño existente.

Para los test unitarios se probará los diferentes métodos por separado. Por lo cual se ejecutarán los test para los métodos CRUD de los perros. Esto para probar un correcto funcionamiento. Entonces se tienen los siguientes Test para perro estáticos.

```
# Test para perros estáticos

def test_create_static_dog():
    db = next(override_get_db())
    setup_db()

    test_dog = StaticDogCreate(
        id_chip=12345,
```

```
        name="Test Dog",
        about="A friendly dog",
        age=5,
        is_vaccinated=True,
        image=None,
        gender=Gender.MALE,
        entry_date=date.today(),
        is_sterilized=True,
        is_dewormed=True,
        operation="None",
    )
    result = create_static_dog(db, test_dog)
    assert result == {"detail": "Perro Permanente creado"}
    teardown_db()


def test_read_all_static_dogs():
    db = next(override_get_db())
    setup_db()

    test_dog_1 = StaticDogCreate(
        id_chip=11111,
        name="Dog 1",
        about="First Dog",
        age=4,
        is_vaccinated=True,
        image=None,
        gender=Gender.MALE,
        entry_date=date.today(),
        is_sterilized=True,
        is_dewormed=True,
        operation="None",
    )
    test_dog_2 = StaticDogCreate(
        id_chip=22222,
        name="Dog 2",
        about="Second Dog",
        age=2,
        is_vaccinated=False,
        image=None,
        gender=Gender.FEMALE,
        entry_date=date.today(),
        is_sterilized=False,
        is_dewormed=True,
        operation="Operation",
    )
    create_static_dog(db, test_dog_1)
    create_static_dog(db, test_dog_2)

    result = read_all_static_dogs(db)
    assert len(result) == 2
    teardown_db()


def test_read_static_dogs_by_id():
    db = next(override_get_db())
    setup_db()

    test_dog = StaticDogCreate(
        id_chip=33333,
```

```
        name="Dog by ID",
        about="Specific Dog",
        age=1,
        is_vaccinated=True,
        image=None,
        gender=Gender.MALE,
        entry_date=date.today(),
        is_sterilized=False,
        is_dewormed=True,
        operation="None",
    )
    create_static_dog(db, test_dog)
    dog = db.query(StaticDog).filter_by(id_chip=33333).first()

    result = read_static_dogs_by_id(db, dog.id)
    assert result.name == "Dog by ID"
    teardown_db()


def test_update_static_dog():
    db = next(override_get_db())
    setup_db()

    test_dog = StaticDogCreate(
        id_chip=44444,
        name="Dog to Update",
        about="Updatable Dog",
        age=6,
        is_vaccinated=False,
        image=None,
        gender=Gender.FEMALE,
        entry_date=date.today(),
        is_sterilized=False,
        is_dewormed=False,
        operation="None",
    )
    create_static_dog(db, test_dog)
    dog = db.query(StaticDog).filter_by(id_chip=44444).first()

    updated_dog = StaticDogCreate(
        id_chip=44444,
        name="Updated Dog",
        about="Updated Description",
        age=7,
        is_vaccinated=True,
        image=None,
        gender=Gender.FEMALE,
        entry_date=date.today(),
        is_sterilized=True,
        is_dewormed=True,
        operation="Updated",
    )
    result = update_static_dog(db, updated_dog, dog.id)
    assert result == {"detail": "Perro Permanente Actualizado"}
    teardown_db()


def test_delete_an_static_dog_by_id():
    db = next(override_get_db())
    setup_db()
```

```
    test_dog = StaticDogCreate(
        id_chip=55555,
        name="Dog to Delete",
        about="Deletable Dog",
        age=3,
        is_vaccinated=False,
        image=None,
        gender=Gender.MALE,
        entry_date=date.today(),
        is_sterilized=True,
        is_dewormed=True,
        operation="None",
    )
    create_static_dog(db, test_dog)
    dog = db.query(StaticDog).filter_by(id_chip=55555).first()

    result = delete_an_static_dog_by_id(db, dog.id)
    assert result is True
    teardown_db()
```

Los demás test de los métodos se encuentran en el Anexo II.

Al ejecutar todos los test del módulo de gestión de perros, se tiene los siguientes resultados presentes en la Figura 7, se observa que pasaron 14 test de los 14 planteados, lo que indica que los métodos funcionan correctamente.

```
============================= test session starts ==============================
collecting ... collected 14 items

test_dog.py::test_create_static_dog PASSED                               [  7%]
test_dog.py::test_read_all_static_dogs PASSED                            [ 14%]
test_dog.py::test_read_static_dogs_by_id PASSED                          [ 21%]
test_dog.py::test_update_static_dog PASSED                               [ 28%]
test_dog.py::test_delete_an_static_dog_by_id PASSED                      [ 35%]
test_dog.py::test_create_adoption_dog PASSED                             [ 42%]
test_dog.py::test_read_all_adoption_dogs PASSED                          [ 50%]
test_dog.py::test_read_adoption_dog_by_id PASSED                         [ 57%]
test_dog.py::test_update_adoption_dog PASSED                             [ 64%]
test_dog.py::test_delete_an_adoption_dog_by_id PASSED                    [ 71%]
test_dog.py::test_adopt_dog PASSED                                       [ 78%]
test_dog.py::test_create_adopted_dog_without_commit PASSED               [ 85%]
test_dog.py::test_read_adopted_dogs_by_id PASSED                         [ 92%]
test_dog.py::test_update_adopted_dog PASSED                              [100%]

======================= 14 passed, 14 warnings in 1.51s ========================

Process finished with exit code 0
```

Figura 7. Resultado de los test unitarios para el módulo de gestión de perros**.**

3.2 Conclusiones

Se concluyó el desarrollo del Backend para el sistema del Club de Bienestar Animal. El componente cumple con todos los requisitos planteados, asegurando una correcta funcionalidad en base a criterios de aceptación y una fácil integración.

Se logró implementar correctamente un módulo enfocado en la gestión de perros el cual permite tener constancia y mantener registros actualizados de la información de los perros actuales, pasados y futuros de la universidad. Además de un punto importante en este módulo el cual es el registro de las adopciones y de ser el caso la remoción de las adopciones. Además, contando col la posibilidad de asociar un dueño a dichos perros que son adoptados, ingresando un nuevo registro de este, o asociarlo a uno ya existente de ser el caso.

Se implementó otro módulo importante el cual ayuda con la gestión de los perros adoptados, permitiendo asegurar el bienestar de los animalitos. Realizando seguimientos

basados en visitas en los cuales se recolectará y almacenará información importante acerca del estado del animal, como lo es una evidencia.

Gracias a la implementación en forma de API RESTful podemos exponer endpoints a los cuales se tendrá acceso para hacer uso del sistema, mediante estos permitiendo una conexión con otros sistemas que, ya sea necesiten la información, actualizar o persistir nueva información. Además, su implementación hecha con Python y el framework enfocado en el desarrollo de APIs, FastApi, ayuda con la rápides y facilidad de mantenimiento del sistema. Además de contar con la herramienta Swagger que permite tener una documentación interactiva o “Documentación viviente” con la cual se puede comprender de mejor manera el funcionamiento de cada uno de los endpoints.

Otra funcionalidad implementada es la gestión de cursos y sus inscripciones, para ayudar y facilitar a la administradora del curso ofertas de forma fácil la información relacionada a estos. Además de recibir solicitudes de personas interesadas, las cuales se permiten controlar en base a cupos en cada curso.

Finalmente se implementaron medidas de seguridad basadas en roles para restringir el acceso a cuentas que no estén autorizadas. Usando Hash de contraseñas y encriptación de datos sensibles como lo son la información de un dueño. Además, además de permitir crear y eliminar usuarios para controlar el acceso. Como también la personalización y actualización de credenciales de acceso para personalizar y evitar el robo del acceso. En conjunto con la Correo para la recuperación de la contraseña para retomar, de igual manera el acceso a este.

3.3 Recomendaciones

- Es importante la mejora de tiempos de respuesta y refactorización de procesos complejos para mejorar la eficiencia del sistema o la busca de otras alternativas al almacenamiento y obtención de imágenes.
- Para futuras integraciones, es posible el desarrollo de módulos que permitan la gestión del inventario de los artículos del club para tener el control de artículos como comida, medicinas, entre otros. Además de módulos que permitan recibir solicitudes o postulaciones para adopciones de los perros y poder gestionarlas desde el mismo sistema.

- Se sugiere la generación de reportes los cuales puedan ser exportados a archivos de documentos los cuales contengan información de reportes del bienestar de los diferentes animales. Así también mejorar y ampliar los roles para permitir una mejor gestión y colaboración.

4 REFERENCIAS BIBLIOGRÁFICAS

[1] Quito Informa, «¿Cuántos animales en condición de calle hay en Quito?,» Quito Informa, 16 2 2024. [En línea]. Available: https://www.quitoinforma.gob.ec/2024/02/16/cuantos-animales-en-condicion-de-calle-hay-en-quito/. [Último acceso: 20 1 2025].

[2] Quito Informa, «Quito contará con una estrategia de control de fauna urbana,» Quito Informa, 26 4 2024. [En línea]. Available: https://www.quitoinforma.gob.ec/2024/04/26/quito-contara-con-una-estrategia-de-control-de-fauna-urbana/. [Último acceso: 20 1 2025].

[3] Amazon, «What is a database?,» AWS, [En línea]. Available: https://aws.amazon.com/es/what-is/database/. [Último acceso: 18 1 2025].

[4] I. B. a. J. R. R. Jacobson, The Unified Modeling Language User Guide, Boston, 2005.

[5] R. Awati, «object-relational mapping (ORM),» The Server Side, 3 2023. [En línea]. Available: https://www.theserverside.com/definition/object-relational-mapping-ORM. [Último acceso: 22 1 2025].

[6] Google, 2022. [En línea]. Available: https://cloud.google.com/learn/what-are-containers?hl=es. [Último acceso: 5 1 2025].

[7] P. A. V. H. a. J. H. R. M. Hong Zhu, «Software unit test coverage and adequacy,» de *Validation, verification, and testing of computer software. Comput. Surv.*, New York, NY, USA, Association for Computing Machinery, 1997 , pp. 366 - 427.

[8] Scrum.org, «Que es Scrum,» Scrum.org, [En línea]. Available: https://www.scrum.org/resources/what-scrum-module. [Último acceso: 18 1 2025].

[9] S. Y. a. H. Halpin, «citeseerx,» 2006. [En línea]. Available: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=1f2ee3831eebfc97bfafd514ca2abb7e2c5c86bb. [Último acceso: 5 1 2025].

[10] Pycharm, «JETBRAINS,» [En línea]. Available: https://www.jetbrains.com/es-es/pycharm/. [Último acceso: 10 1 2025].

[11] EDUCBA, «What is XAMPP?,» EDUCBA, 5 4 2023. [En línea]. Available: https://www.educba.com/what-is-xampp/. [Último acceso: 15 1 2025].

[12] G. BUSTOS, «HOSTINGER,» 10 1 2023. [En línea]. Available: https://www.hostinger.es/tutoriales/que-es-github. [Último acceso: 10 1 2025].

[13] FastApi, «FastApi,» [En línea]. Available: https://fastapi.tiangolo.com/. [Último acceso: 23 1 2025].

[14] K. S. Rubin, Essential Scrum: a practical guide to the most popular agile process, Michigan: Addison-Wesley, 2012.

5 ANEXOS

ANEXO I. Historias de usuario (HU)

ANEXO II. Enlaces

ANEXO I

Historia épica

Módulo de Gestión de Cursos (GC)

Como Administradora quiero gestionar los cursos que el club puede ofrecer para que presente la información detallada y actualizada de estos.

Historias de usuario

		HU
Identificador: GC-1	**Usuario:** Administradora	
Nombre: Creación de cursos		
Prioridad: Media	**Riesgo:** Bajo	
Estimados: 5	**Iteración:** 1	
Descripción: Como Administradora quiero crear nuevos cursos para poder ofertar estos al público general.		
Criterios de aceptación: - Se ingresará el nombre del curso, descripción, cantidad de cupos, precio y horarios. - Campos como el nombre, cantidad de cupos y horarios son obligatorios. - El curso se persistirá la información en la db		

		HU
Identificador: GC-2	**Usuario:** Usuario del sistema	
Nombre: Obtener información de cursos		
Prioridad: Media	**Riesgo:** Bajo	
Estimado: 3	**Iteración:** 1	
Descripción: Como usuario del sistema quiero saber la información de los cursos disponibles en el sistema para que pueda tomar decisiones informadas sobre a qué curso inscribirme.		
Criterios de aceptación: - Se retornará una lista con la información general de todos los cursos creados. - Se presentará la información detallada y completa de un curso en específico		

		HU
Identificador: GC-3	**Usuario:** Administradora	
Nombre: Actualizar Cursos		
Prioridad: Media	**Riesgo:** Bajo	
Estimado: 5	**Iteración:** 2	
Descripción: Como administradora del sistema quiero actualizar los campos recibidos de los cursos para que estos ya no presenten información incompleta o errónea.		
Criterios de aceptación: - Recibir los nuevos cambios mediante un archivo json. - Persistir los cambios. - Mostar un mensaje de confirmación de los cambios.		

		HU
Identificador: GC-4	**Usuario:** Administradora	
Nombre: Eliminar curos		
Prioridad: Media	**Riesgo:** Bajo	
Estimado: 3	**Iteración: 4**	
Descripción: Como Administradora quiero eliminar los cursos que ya no estén disponible para mantener los registros de estos actualizados.		
Criterios de aceptación: - Se recibirá el id del curso a eliminar - Se realizaron los cambios. - El curso ya no estará disponible		

Historia épica

Módulo de Sesión y Acceso (IS)

Como usuario quiero tener un mantener un control del acceso mediante credenciales como el nombre de usuario y contraseña, para que se restrinja el acceso.

Historias de usuario

		HU
Identificador: IS-1	**Usuario:** Usuario del sistema	
Nombre: Generación de Token de autenticación		
Prioridad: Alta	**Riesgo:** Alta	
Estimado: 8	**Iteración:** 1	
Descripción: Como usuario quiero acceder mediante usuario u contraseña para que se autentique y tener permisos de acceder y hacer uso de los módulos.		
Criterios de aceptación: - Se autorizará mediante usuario y contraseña. - Se generará un token que valide al usuario y contenga información importante como el rol. - Se validará el token y este expirará después de 30 minutos de su creación.		

		HU
Identificador: IS-2	**Usuario:** Usuario general del sistema	
Nombre: Correo para la recuperación de la contraseña		
Prioridad: Media	**Riesgo:** Media	
Estimado: 5	**Iteración:** 1	
Descripción: Como usuario general del sistema quiero recibir un correo que permita reestablecer mi contraseña para volver a tener acceso.		
Criterios de aceptación: - Se recibirá un correo con una estructura válida de una cuenta asociada. - En caso de existir el correo se enviará uno con un código. - El código se validará y se extenderá su duración. - Se volverá a validar el código en conjunto con una nueva contraseña y se la actualizará a la cuenta del correo asociado.		

Historia épica

Módulo de Gestión de Perros (GP)

Como administradora del sistema, quiero contar con un apartado completo que me permita registrar, consultar, actualizar y eliminar información sobre perros permanentes, temporales y adoptados, para que pueda administrar de forma eficaz y centralizada los datos de los perros y asegurarme de que la información esté actualizada y correctamente clasificada.

Historias de usuario

		HU
Identificador: GP-1	**Usuario:** Administradora	
Nombre: Registrar perro permanente		
Prioridad: Alta	**Riesgo:** Alta	
Estimado: 5	**Iteración:** 1	
Descripción: Como administradora quiero registrar un perro permanente para mantener un registro actualizado de los perros comunitarios.		
Criterios de aceptación: - Se debe recibir el nombre, edad, género, vacunado, esterilizado, desparasitado, número de chip, fecha de ingreso, operaciones y foto. - La información debe persistirse.		

		HU
Identificador: GP-2	**Usuario:** Administradora	
Nombre: Registrar perro temporal		
Prioridad: Alta	**Riesgo:** Alta	
Estimado: 5	**Iteración: 1**	
Descripción: Como administradora quiero registrar un perro temporal para mantener un registro actualizado de los perros disponibles de adopción.		
Criterios de aceptación: - Se debe recibir el nombre, edad, género, vacunado, esterilizado, desparasitado, número de chip, fecha de ingreso, operaciones y foto. - La información debe persistirse.		

		HU
Identificador: GP-3	**Usuario:** Administradora	
Nombre: Registrar perro adoptado		
Prioridad: Alta	**Riesgo:** Alta	
Estimado: 5	**Iteración:** 3	
Descripción: Como administradora quiero registrar un perro adoptado para mantener un registro actualizado de los perros disponibles de adopción de la universidad.		
Criterios de aceptación:		

-se debe recibir la fecha de adopción, nombre del adoptante, dirección y teléfono y un id de perro estático. - Se puede registrar una adopción con un dueño existente. - Se debe eliminar el perro temporal. - Se debe persistir el perro adoptado junto a su dueño

		Historia de usuario
Identificador: GP-4	**Usuario:** Administradora	
Nombre: Consultar lista de perros permanentes		
Prioridad: Alta	**Riesgo:** Media	
Estimado: 3	**Iteración:** 1	
Descripción: Como administradora obtener la información de los perros permanentes para acceder fácilmente a la información almacenada y gestionarlos.		
Criterios de aceptación: Se debe obtener un json con la información del perro el nombre, edad, género, si esta vacunado, esterilizado y desparasitado, número de chip, fecha de ingreso y si tiene operaciones.		

		Historia de usuario
Identificador: GP-5	**Usuario:** Administradora	
Nombre: Consultar lista de perros temporales		
Prioridad: Alta	**Riesgo:** Media	
Estimado: 3	**Iteración:** 1	
Descripción: Como administradora obtener la información de los perros temporales para acceder fácilmente a la información almacenada y gestionarlos.		
Criterios de aceptación: Se debe obtener un json con la información del perro el nombre, edad, género, si esta vacunado, esterilizado y desparasitado, número de chip, fecha de ingreso y si tiene operaciones.		

		HU
Identificador: GP-6	**Usuario:** Administradora	
Nombre: Consultar lista con perros adoptados		
Prioridad: Alta	**Riesgo:** Media	
Estimado: 3	**Iteración:** 1	
Descripción: Como administradora obtener la información de los perros adoptados y sus dueños para acceder fácilmente a la información almacenada y gestionarlos.		
Criterios de aceptación: Se debe obtener un json con la información del perro el nombre, edad, género, si esta vacunado, esterilizado y desparasitado, número de chip, fecha de ingreso y si tiene operaciones, además de la información del dueño con su nombre, dirección y teléfono.		

		HU
Identificador: GP-7	**Usuario:** Administradora	
Nombre: Actualizar el registro del perro permanente		

Prioridad: Alta	**Riesgo:** Alta
Estimado: 5	**Iteración:** 2
Descripción: Como administradora quiero actualizar el registro de un perro permanente para corregir la información o agregar detalles nuevos.	
Criterios de aceptación: Se validará si existe el perro. El sistema registrar los cambios del como nombre, edad, género, vacunado, esterilizado, desparasitado, número de chip, fecha de ingreso, operaciones y foto. Los cambios deben reflejarse en la base de datos. Se deben mostrar mensajes de confirmación.	

		HU
Identificador: GP-8	**Usuario:** Administradora	
Nombre: Actualizar perro temporal		
Prioridad: Alta		**Riesgo:** Alta
Estimado: 5		**Iteración:** 2
Descripción: Como administradora quiero actualizar los datos de un perro temporal para corregir la información o agregar detalles nuevos.		
Criterios de aceptación: Se validará si existe el perro. Se deben recibir los cambios con la información del perro, su nombre, edad, género, vacunado, esterilizado, desparasitado, número de chip, fecha de ingreso, operaciones y foto. Los cambios deben reflejarse en la base de datos. Se deben mostrar mensajes de confirmación.		

		HU
Identificador: GP-9	**Usuario:** Administradora	
Nombre: Actualizar perro adoptado		
Prioridad: Alta		**Riesgo:** Alta
Estimado: 5		**Iteración:** 3
Descripción: Como administradora quiero actualizar los datos de un perro adoptado para corregir información o agregar nuevos detalles.		
Criterios de aceptación: Se validará si existe el perro. Se deben recibir los cambios en los campos como nombre, edad, género, vacunado, esterilizado, desparasitado, número de chip, fecha de ingreso, operaciones y foto. Los cambios deben reflejarse inmediatamente en la base de datos Se deben mostrar mensajes de confirmación.		

		HU
Identificador: GP-10	**Usuario:** Administradora	
Nombre: Eliminar registro de perro permanente		
Prioridad: Media		**Riesgo:** Alta
Estimado: 3		**Iteración:** 2
Descripción: Como administradora quiero quitar un perro permanente para mantener actualizado el registro de perros.		

Criterios de aceptación: Se validará si existe el perro. El perro eliminado debe desaparecer de la base de datos

		HU
Identificador: GP-11	**Usuario:** Administradora	
Nombre: Eliminar registro de perro temporal		
Prioridad: Media	**Riesgo:** Alta	
Estimado: 3	**Iteración:** 2	
Descripción: Como administradora quiero retirar un perro temporal para mantener un registro actualizado.		
Criterios de aceptación: Se validará si existe el perro. El perro eliminado debe desaparecer de la db.		

		HU
Identificador: GP-12	**Usuario:** Administradora	
Nombre: Eliminar registro de perro adoptado		
Prioridad: Media	**Riesgo:** Alta	
Estimado: 5	**Iteración:** 3	
Descripción: Como administradora quiero eliminar el registro de adopción para reflejar casos de devoluciones o cancelaciones.		
Criterios de aceptación: Se validará que exista el perro. El dueño debe eliminarse de la base de datos. El perro se moverá a perros temporales.		

		HU
Identificador: GP-13	**Usuario:** Usuario del sistema	
Nombre: Consultar perros en base a su id		
Prioridad: Alta	**Riesgo:** Media	
Estimado: 5	**Iteración:** 3	
Descripción: Como administradora quiero obtener la información un perro en específico para no obtener información no relacionada.		
Criterios de aceptación: Se validará si el perro solicitado existe. Se retornará toda la información del perro solicitado.		

Historia épica

Módulo de Visitas y Seguimiento (VS)

Como administradora del sistema registrar y gestionar las visitas realizadas a los perros adoptados para llevar un control detallado de su bienestar y estado de salud.

HU

		HU
Identificador: VS-1	**Usuario:** Administradora	
Nombre: Registrar visita		
Prioridad: Alta	**Riesgo:** Alta	
Estimado: 3	**Iteración:** 4	
Descripción: Como administradora quiero registrar una visita realizada a un perro adoptado para documentar observaciones, tratamientos o cualquier detalle relevante de la interacción.		
Criterios de aceptación: Se deben recibir la información de una visita, la fecha, evidencia y observaciones. Debe estar asociado al perro correspondiente. Se retornar una confirmación.		

		HU
Identificador: VS-2	**Usuario:** Administradora	
Nombre: Obtener lista Visitas		
Prioridad: Alta	**Riesgo:** Alta	
Estimado: 3	**Iteración:** 4	
Descripción: Como administradora obtener las visitas realizadas para tener un registro global y accesible de las actividades de seguimiento.		
Criterios de aceptación: Se debe retornar la información del perro, nombre del dueño asociadas a la visita registrada.		

		HU
Identificador: VS-3	**Usuario:** Administradora	
Nombre: Obtener visitas de un Perro		
Prioridad: Alta	**Riesgo:** Media	
Estimado: 3	**Iteración:** 4	
Descripción: Como administradora quiero filtrar y obtener las visitas realizadas a un perro específico para hacer un seguimiento detallado de su evolución y cuidado.		
Criterios de aceptación: Se valida si el perro existe. Se valida si tiene visitas. Se retornarán las visitas asociadas al perro.		

		HU
Identificador: VS-4	**Usuario:** Administradora	
Nombre: Actualizar visita		
Prioridad: Alta	**Riesgo:** Alta	
Estimado: 5	**Iteración:** 4	
Descripción: Como administradora quiero actualizar una visita previamente registrada para corregir errores o añadir información adicional.		
Criterios de aceptación: El sistema debe recibir los cambios en los campos: fecha, evidencia y observaciones. Debe estar asociado al perro correspondiente.		

Se debe mostrar mensajes de confirmación. Los cambios deben reflejarse correctamente.

		HU
Identificador: VS-5	**Usuario:** Administradora	
Nombre: Eliminar visita		
Prioridad: Media		**Riesgo:** Media
Estimado: 3		**Iteración:** 4
Descripción: Como administradora quiero actualizar una visita previamente registrada para corregir errores o añadir información adicional.		
Criterios de aceptación: Se deben recibir los cambios en los campos: fecha, evidencia y observaciones. Debe estar asociado al perro correspondiente. Se mostrará un mensaje de confirmación. Los cambios deben reflejarse correctamente.		

		HU
Identificador: VS-6	**Usuario:** Administradora	
Nombre: Actualizar información de dueño		
Prioridad: Baja		**Riesgo:** Baja
Estimado: 5		**Iteración:** 4
Descripción: Como administradora quiero actualizar la información del dueño previamente registrada para corregir errores o mantener actualizada la información de contacto.		
Criterios de aceptación: Se deben recibir los cambios en la información del dueño Debe estar asociado al perro correspondiente. Se mostrará un mensaje de confirmación. Los cambios deben reflejarse correctamente		

Módulo de Configuración Personal (CP)

		HU
Identificador: CP-1	**Usuario:** Usuario del sistema	
Nombre: Actualizar credenciales		
Prioridad: Baja		**Riesgo:** Alta
Estimado: 3		**Iteración:** 4
Descripción: Como usuario quiero actualizar las credenciales genéricas a unas nuevas para tener control sobre estas mismas.		
Criterios de aceptación: Se reciben los cambios en los campos nombre de usuario y contraseña. Se mostrará el mensaje de confirmación. Se reflejan los cabios en la base de datos		

Historia épica

Componente para la Administración de Usuarios (GU)

Como administradora quiero gestionar los usuarios dando y revocando su acceso para controlar el acceso al sistema.

HU

		HU
Identificador: GU-1	**Usuario:** Administradora	
Nombre: Generar usuario		
Prioridad: Media	**Riesgo:** Baja	
Estimado: 5	**Iteración:** 3	
Descripción: Como administradora quiero crear nuevos usuarios para permitirles acceder al sistema y realizar tareas asignadas.		
Criterios de aceptación: El usuario debe ser un administrador. Se deben recibir los campos de correo electrónico y rol. Se enviará un correo con las credenciales al email proporcionado.		

		HU
Identificador: GU-2	**Usuario:** Administradora	
Nombre: Obtener Usuarios Creados		
Prioridad: Baja	**Riesgo:** Baja	
Estimado: 3	**Iteración:** 3	
Descripción: Como administradora quiero obtener todos los usuarios registrados para administrar las cuentas activas en el sistema.		
Criterios de aceptación: Se debe tener el rol de administrador. Se retorna el id y nombre de usuario existentes.		

		HU
Identificador: GU-3	**Usuario:** Administradora	
Nombre: Eliminar usuarios		
Prioridad: Baja	**Riesgo:** Baja	
Estimado: 5	**Iteración:** 3	
Descripción: Como Administradora quiero eliminar usuarios registrados para revocar el acceso al sistema.		
Criterios de aceptación: Se debe tener el rol de administrador. El sistema no debe permitir eliminar el usuario de la sesión La eliminación debe reflejarse en la base de datos.		

Historia épica

Módulo de Gestión de Inscripciones (GI)

Como Administradora quiero gestionar las inscripciones hechas a cursos ofertados por el club para obtener o eliminar la información de contacto de los interesados

Historias de usuario

		HU
Identificador: GI-1	**Usuario:** Interesado en el curso	
Nombre: Inscripciones a un curso específico		
Prioridad: Media	**Riesgo:** Baja	
Estimado: 8	**Iteración:** 5	
Descripción: Como interesado en los cursos quiero registrar una inscripción en un curso para apartar un cupo en esta.		
Criterios de aceptación: Se recibirá la información de interesado con su información de contacto. Se validará si existen cupos disponibles.		

		HU
Identificador: GI-2	**Usuario:** Administradora	
Nombre: Eliminar inscripciones		
Prioridad: Baja	**Riesgo:** Media	
Estimado: 5	**Iteración:** 5	
Descripción: Como Administradora quiero eliminar inscripciones de antiguos estudiantes o solicitudes invalidas para tener un control correcto de estas.		
Criterios de aceptación: Se recibirá el id de la inscripción Se eliminará la inscripción y se reflejará en la base de datos.		

ANEXO II

Enlace al repositorio de GitHub: https://github.com/TheDanielV/PoliPerritos.git